PhishGuard AI: Intelligent Phishing Detection System

Project documentation | Applied Cryptography and Network Security (CNS) Design Challenge

1. Overview

PhishGuard AI is a web-based tool that checks whether a URL is legitimate or a phishing attempt. A user pastes a URL into a dashboard, the system extracts 17 features from the URL text, and a Random Forest model returns a verdict with a risk score from 0 to 100%.

The system looks only at the URL itself (lexical analysis). It does not visit the page or call outside services, so results are fast and safe to run on suspicious links.

2. Problem Statement

- Phishing attacks trick users into giving away credentials or downloading malicious files.
- Attackers keep changing domains and hiding URLs with subdomains, symbols and look-alike names.
- Blacklists are slow to update, which leaves users exposed to new phishing sites.

3. Proposed Solution

1. Validate and clean the URL entered by the user.
2. Extract 17 lexical features (length, subdomains, special characters, HTTPS and more).
3. Pass the features to a Random Forest classifier.
4. Show the result as Legitimate or Potential Phishing, with a risk score and a breakdown of the features.

4. Key Features

- Real-time URL scanning
- 17-feature lexical extractor
- Risk score from 0 to 100%

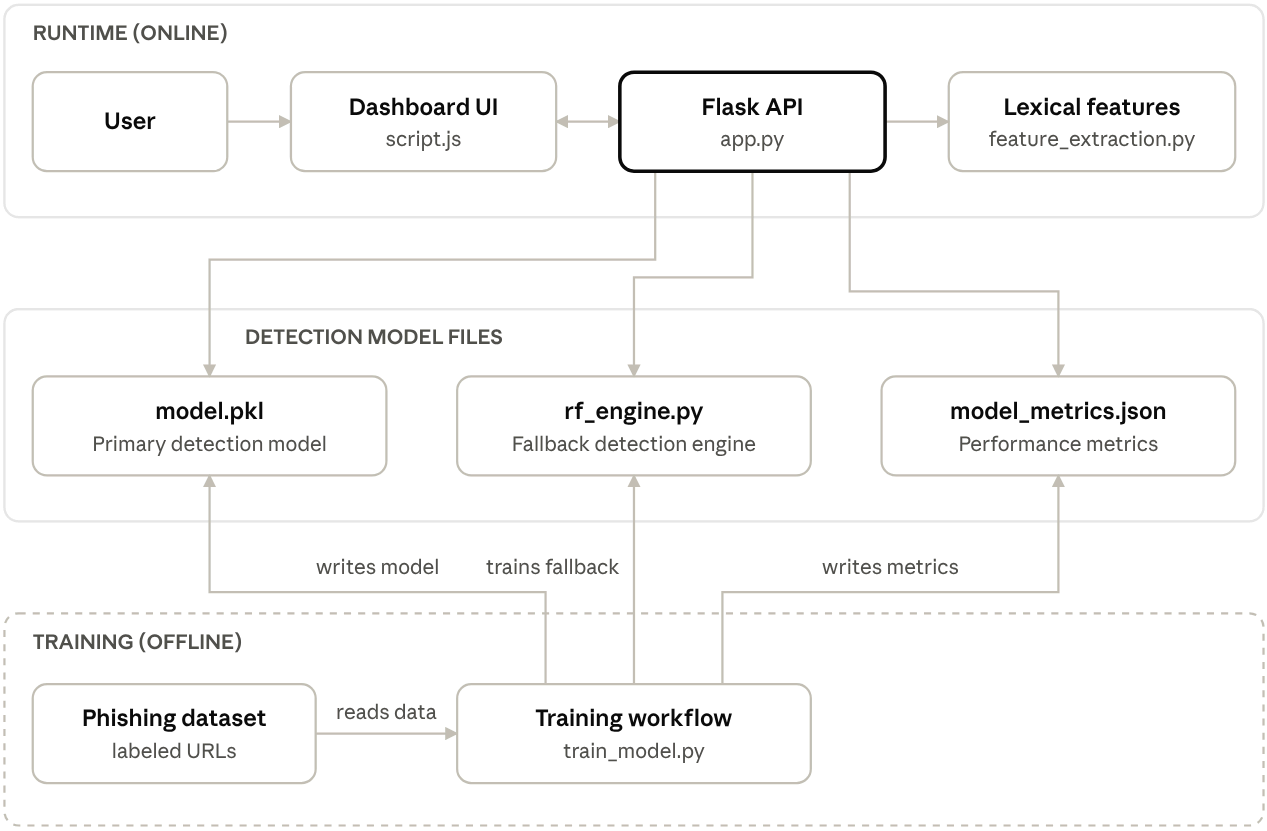
- Feature breakdown that explains why a URL was flagged
- Model performance panel (accuracy, precision, recall, F1, confusion matrix)
- Quick-test sample URLs for demos
- Fallback engine in pure Python if scikit-learn cannot be used

5. System Architecture

The system has three parts: offline model training, the online runtime (dashboard and API), and the saved model files that connect them.

Part	File	Role
Training workflow	train_model.py	Reads the dataset, trains the model, writes model.pkl and model_metrics.json
Dashboard UI	script.js	Takes the URL from the user, calls the API, shows the result
Flask API	app.py	Serves the dashboard and exposes /predict, /health and /api/metrics
Lexical features	feature_extraction.py	Converts a URL into the 17 features
Trained model	model.pkl	Main detection model
Python Forest	rf_engine.py	Fallback detection engine
Evaluation metrics	model_metrics.json	Saved performance numbers shown on the dashboard

The dashboard talks to one Flask API, which uses model files built offline



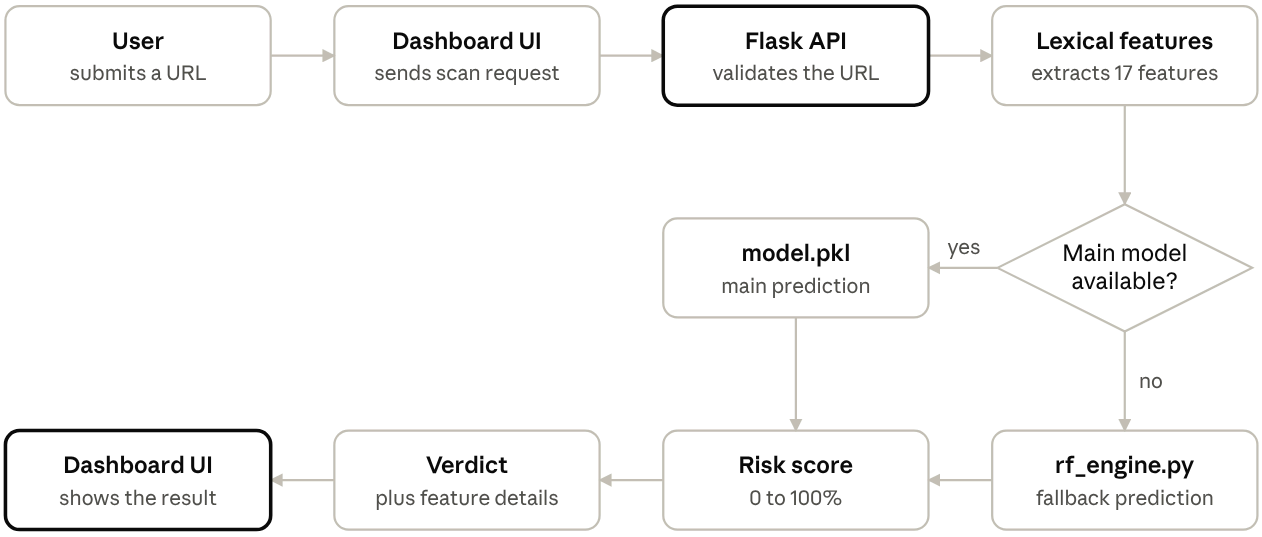
system architecture · 3 zones, 9 components

The dashboard and API run online, the saved model files sit between them, and the offline training workflow writes those files.

Runtime flow

1. The user submits a URL in the dashboard.
2. The dashboard sends a scan request to the Flask API.
3. The API extracts the 17 lexical features from the URL.
4. The API loads the trained model and predicts. If needed, it uses the Python Forest fallback.
5. The API returns the prediction, risk score and feature details to the dashboard.

Every scan ends in a risk score and a verdict, whichever model predicts



runtime flow · 9 steps, 1 decision

If the main model is not available, the Python Forest fallback makes the prediction instead.

Training flow

1. train_model.py reads the labeled URLs from the phishing dataset.
2. It trains the Random Forest model and a fallback model.
3. It writes model.pkl and model_metrics.json.

6. API Endpoints

Endpoint	Method	Purpose
/	GET	Serves the dashboard
/health	GET	Reports whether the backend and model are running
/api/metrics	GET	Returns the saved evaluation metrics
/predict	POST	Takes a URL and returns the verdict, risk score and feature details

7. Dataset

- Name: PhiUSIIL Phishing URL Dataset (UCI Machine Learning Repository)
- Size: 235,795 rows and 56 columns
- Labels: 1 = Legitimate (134,850 samples), 0 = Phishing (100,945 samples)
- Quality: no missing values and no duplicate records

8. Machine Learning Model

- Algorithm: Random Forest Classifier
- Trees: 100
- Random state: 42
- Train/test split: 80% / 20%, stratified by class

9. The 17 Features

#	Feature	Description
1	URLLength	Total characters in the URL
2	DomainLength	Characters in the domain
3	IsDomainIP	1 if the host is an IP address
4	NoOfSubDomain	Number of subdomains
5	HasObfuscation	1 if the URL contains @, % or // tricks
6	NoOfObfuscatedChar	Count of obfuscation symbols
7	ObfuscationRatio	Obfuscation symbols divided by URL length
8	NoOfLettersInURL	Count of letters
9	LetterRatioInURL	Letters divided by URL length
10	NoOfDegitsInURL	Count of digits
11	DegitRatioInURL	Digits divided by URL length
12	NoOfEqualsInURL	Count of =
13	NoOfQMarkInURL	Count of ?

#	Feature	Description
14	NoOfAmpersandInURL	Count of &
15	NoOfOtherSpecialCharsInURL	Other special characters
16	SpacialCharRatioInURL	Special characters divided by URL length
17	IsHTTPS	1 if the URL uses HTTPS

10. Evaluation Results

Metric	Value
Accuracy	99.33%
Precision	99.02%
Recall	99.86%
F1 Score	99.44%

Confusion matrix (rows = actual, columns = predicted):

	Predicted Phishing	Predicted Legitimate
Actual Phishing	976	14
Actual Legitimate	2	1408

11. Technology Stack

- Backend: Python 3.12, Flask
- Machine learning: Random Forest, scikit-learn, joblib
- Feature extraction: Python standard library (urllib.parse, re, ipaddress)
- Frontend: HTML, CSS and JavaScript (fetch API)
- Dataset: PhiUSIIL Phishing URL Dataset

12. Project Structure

Path	Contents
backend/app.py	Flask server and API
backend/feature_extraction.py	17-feature URL parser
backend/rf_engine.py	Fallback Random Forest engine
backend/train_model.py	Training, evaluation and export
backend/model.pkl	Trained model
backend/model_metrics.json	Saved metrics
data/phishing_dataset.csv	Dataset
frontend/index.html, style.css, script.js	Dashboard
run.py and start.bat	One-command launchers
requirements.txt	Python dependencies

13. How to Run

1. Install Python 3.12.
2. Install dependencies: pip install -r requirements.txt
3. Start the project: python run.py (or double-click start.bat on Windows).
4. Open http://127.0.0.1:5000 in a browser.

If model.pkl is missing, run.py trains the model automatically before starting the server.

14. Demo Steps

1. Check that the top-right badge shows the ML engine is active.
2. Look at the Model Performance Metrics section.
3. Click the Academic Site sample (https://www.uni-mainz.de) and press Check URL. The result should be Legitimate with a low risk score.

4. Click the Phishing Sample (http://secure-login.paypal.account-verify.com/login?id=999&auth=fail) and press Check URL. The result should be Potential Phishing with a high risk score.

15. Limitations

- Only the URL text is analyzed. Page content, domain age and certificates are not checked.
- Predictions are probabilities, not guarantees, and should be used with other security measures.
- The evaluation is based on one dataset, so results may differ on other data.

16. Future Scope

- Browser extension for automatic scanning before a page loads
- Threat intelligence feeds such as VirusTotal and Google Safe Browsing
- Webpage content analysis (forms, hidden password fields, redirects)
- Domain age and SSL certificate checks